

Síndrome do X Frágil

**Marina Cella Scarpari, Lucas Tian Le Wu, Christian Hideyuki Sasaoka,
Eduardo dos Santos Granha, Thiago Marcos Burtet**

A Síndrome do X Frágil é uma condição genética ainda pouco conhecida e difundida, que afeta o desenvolvimento intelectual, o comportamento e provoca atrasos na fala. Devido a isso, é frequentemente confundida com transtorno do espectro autista, levando a encaminhamentos equivocados para o teste genético confirmatório.

Um exame de custo elevado, financiado por organizações não-governamentais. Este trabalho apresenta uma plataforma web de triagem clínica que organiza o fluxo entre a suspeita e o teste, com cadastro multiprofissional, escore ponderado pelo sexo do paciente, anexos de evidência visual, rastreamento de familiares e relatórios temporais.

A arquitetura adota Node.js e SQLite com migrations versionadas, React e Vite no cliente. Neste relatório, discutem-se as decisões de projeto, o modelo de dados, o algoritmo de cálculo do escore e a cobertura dos requisitos funcionais e não-funcionais especificados. O sistema foi executado em Apple M5 com macOS Tahoe (Versão 26.5.1), apresentando também pleno funcionamento em Windows 11 Home (Versão 25H2) — ambiente no qual se destaca apenas a necessidade de instalar o Node.js na versão 22.xx.

1. O usuário final

O sistema foi projetado pensando em três classes de usuários:

1.1 Profissional: médicos, pedagogos, psicólogos, fonoaudiólogos, voluntários da ONG, terapeutas ocupacionais, assistentes sociais que possuam vínculo direto com a criança e possam observar suas características.

1.2 Administrador: responsável pela ONG, com permissões para gerenciar contas de profissionais, ajustar o limiar do score e consultar o registro de auditoria.

1.3 Familiar: não acessa a plataforma diretamente, mas suas respostas podem ser registradas por um profissional via ligação telefônica, com explícita identificação da relação com o paciente em cada sessão.

2. Arquitetura do sistema

A plataforma é estruturada como um monorepo, uma estratégia de arquitetura de software onde o código-fonte de múltiplos projetos ou pacotes é armazenado e versionado em um único repositório. Sob o gerenciamento de npm workspaces, dividido em dois pacotes: server (API REST em Node.js) e client (interface em React). Um script de inicialização (scripts/setup.js) unifica a aplicação das migrations e a execução dos dois processos sob um único comando npm run start.

2.1 SQLite com migrations

A documentação inicial do projeto previa o uso de MySQL operado via MySQL Workbench, mas após tentativas de conexão do backend com o front e conversas com os monitores da disciplinas, exploramos a reformulação por SQLite gerenciado por Knex.js devido à ausência de dependência externa. O banco de dados é um único arquivo no sistema de arquivos do servidor, dispensando processo separado, configuração de porta e usuários. Versionamento do esquema. Cada alteração de estrutura é registrada como uma migration numerada e reversível em server/migrations/. O comando npm run migrate aplica somente as pendentes, e o histórico vive sob controle de versão. Backup facilitado consiste em copiar um arquivo e caso seja necessário restaura-lo, é possível sobrescrever. Caso o projeto realmente se torne um site real e seja necessária a troca eventual por PostgreSQL ou MySQL a substituição é simplificada pelo driver no knexfile.js, as migrations permanecem intactas.

2.2 Ferramentas e linguagens utilizadas:

JavaScript/JSX - para frontend e backend, descobrimos o JSX (JavaScript Syntax Extension) que é uma extensão de sintaxe para a linguagem JavaScript. E

possibilita escrever elementos com uma estrutura visual semelhante ao HTML diretamente dentro de seus arquivos e scripts JavaScript.

Node.js $\geq 18.0.0$: Compila e roda o código JavaScript, sendo necessária a versão 18 ou superior. Para execução em Windows 11 Home (versão 25H2), é explicitamente recomendada a versão 22.xx. npm (Node Package Manager): Ferramenta que acompanha o Node.js para gerenciar, baixar e instalar as dependências. É o gerenciador que viabiliza módulos interligados dentro de um único repositório, utilizando a própria estrutura do npm para unificar o ecossistema do projeto.

2.2.1 Frontend (lado do usuário - Client):

React 18.3.1 é a biblioteca principal para a construção da interface do usuário baseada em componentes. O React cuida da renderização, atualizando apenas as partes que sofreram alterações de dados sem precisar recarregar a página inteira.

React Router DOM 6.27.0 ferramenta responsável por gerenciar a navegação e as rotas da aplicação web. Ela permite que o usuário navegue por diferentes "páginas" e telas do sistema sem que o navegador precise fazer uma nova requisição do zero ao servidor.

Vite 5.4.10 (bundler) atua como o motor de desenvolvimento e empacotador do projeto. Ele substitui ferramentas mais antigas (como o Webpack), oferecendo um servidor local extremamente rápido para programar e compilando o código final de forma otimizada para produção.

Tailwind CSS 3.4.14 (styling) para estilização baseado em classes. Em vez de escrever arquivos CSS separados, estilizamos os elementos diretamente no HTML/JSX combinando classes prontas (como flex, pt-4, text-center), o que acelera drasticamente o desenvolvimento visual.

Framer Motion 11.11.9 (animations) biblioteca de animações pronta para o React. Ela facilita a criação de transições fluidas, animações de entrada e saída de elementos e interações como arrastar ou passar o mouse, tornando a experiência do usuário mais dinâmica.

Axios 1.7.7 cliente HTTP utilizado para fazer a ponte entre o Frontend e o Backend e APIs externas. É através dele que o projeto envia e recebe dados de forma

assíncrona, lidando com requisições como login, busca e salvamento de informações.

Lucide React 0.453.0 (icons) pacote que disponibiliza um vasto catálogo de ícones modernos, limpos e customizáveis em formato SVG. Eles são tratados diretamente como componentes React, facilitando a alteração de tamanho, cor e espessura das linhas.

2.2.2 Backend (Servidor):

Express 4.21.0 (web framework): serve como a estrutura principal do Backend. Ele gerencia as rotas da API, escuta as requisições HTTP vindas do Frontend (como requisições GET e POST), controla os middlewares e envia as respostas de volta ao usuário.

SQLite 3 via better-sqlite3 11.3.0 (database): É o banco de dados relacional escolhido para armazenar as informações do sistema. Por ser um banco baseado em arquivo (serverless), ele dispensa uma infraestrutura complexa de servidor separado. O pacote better-sqlite3 é o driver que conecta o Node.js a esse banco, oferecendo uma execução de queries rápida e síncrona.

Knex.js 3.1.0 (query e migrations): É um construtor de consultas SQL (Query Builder) para JavaScript. Em vez de escrever código SQL é possível utilizar funções JavaScript para interagir com o banco de dados. Ele também gerencia o histórico de alterações da estrutura do banco através de migrations, garantindo que a modelagem das tabelas seja idêntica em qualquer ambiente de desenvolvimento.

Bcryptjs 2.4.3 (password hashing): É a biblioteca responsável pela segurança e criptografia de dados sensíveis. Utilizada especificamente para gerar hashes de senhas antes de salvá-las no banco de dados, garantindo que, mesmo em caso de vazamento, as senhas originais dos usuários não possam ser lidas ou descriptografadas.

3. Modelagem de dados

Após a visita técnica ao instituto, notamos a necessidade de mudanças significativas na estrutura inicial do banco de dados. Inicialmente, nossas tabelas estavam focadas em consultas médicas e na mensuração de prioridade pelos profissionais de saúde. No entanto, ficou evidente que as decisões administrativas e o controle do processo deveriam permanecer sob responsabilidade do instituto, já que é ele quem gerencia os recursos, define prioridades e conduz os encaminhamentos para os testes genéticos.

Além disso, demos mais autonomia administrativa ao instituto, garantindo que o poder de decisão permaneça com a equipe responsável pela gestão do processo. Como o instituto administra o uso inteligente dos recursos, define quais casos devem avançar para testes genéticos e acompanha os encaminhamentos, tornou-se prioridade estruturar um banco de dados voltado ao controle, à rastreabilidade, à segurança da informação e à auditoria das ações realizadas.

Dessa forma, o sistema deixa de ser apenas um apoio ao atendimento médico e passa a funcionar como uma ferramenta de gestão institucional. Essa mudança permite maior organização dos dados, controle de acesso, acompanhamento das decisões tomadas e transparência em todo o fluxo, fortalecendo a autonomia do instituto e garantindo que os processos sejam conduzidos de forma mais segura, eficiente e auditável.

4. interface

A interface organiza-se em três zonas distintas: a página inicial pública, o fluxo autenticado para profissionais e o painel administrativo

Rota	Conteúdo
/	landing page, descrição da síndrome e CTAs
/login, /cadastro	autenticação e auto cadastro de profissionais
/dashboard	indicadores agregados e encaminhamentos recentes
/pacientes	busca por nome ou CPF, listagem com situação atual
/pacientes/novo	formulário de cadastro com verificação de CPF em tempo real
/pacientes/:id	prontuário completo: dados, família, scores, anexos
/pacientes/:id/score	assistente de aplicação do score com preview em tempo real
/relatorios	encaminhamentos por intervalo, com exportação pra CSV
/admin	configurações, profissionais e log de auditoria (apenas admin)

5. Cálculo do Score

O cálculo do Score é a métrica central de decisão entre acompanhamento e encaminhamento para o teste genético. Sua lógica está implementada no módulo `server/src/score-engine.js`. Criamos a diferenciação dos pesos através do material disponibilizado, se o sistema de fato for para produção, é um ponto que precisa ser lapidado, para evitar possíveis encaminhamentos equivocados.

Sintoma	Categoria	Peso	Sexo
Face prolongada / orelhas grandes	física	3	M, F
Macroorquidismo	física	4	M
Hipermobilidade articular	física	2	M, F
Deficiência intelectual	cognitiva	3	M, F
Dificuldade de aprendizagem	cognitiva	2	M, F
Atraso na fala	cognitiva	2	M, F
Déficit de atenção	cognitiva	2	M, F
Hiperatividade	comportamental	2	M, F
Movimentos repetitivos	comportamental	2	M, F
Evita contato visual	comportamental	1,5	M, F
Evita contato físico	comportamental	1	M, F
Agressividade / ansiedade social	comportamental	1	M, F

Tabela de referência utilizada

Diferentemente de um questionário pontual, o sistema permite que múltiplas sessões de score sejam aplicadas ao mesmo paciente ao longo do tempo. Cada sessão registra de forma bem clara quem está julgando as características do paciente, sendo possível:

- profissional: o próprio avaliador no consultório;
- familiar_cadastrado — um membro da família já cadastrado no prontuário;
- familiar_telefone — ligação telefônica conduzida pelo Instituto com a família;
- outro — terceiros (professor, terapeuta externo) em campo livre.

Na entrevista com o instituto, essa distinção se mostrou clinicamente relevante: comportamentos sociais variam significativamente conforme o adulto que acompanha a criança no momento da observação. Ao guardar as sessões separadamente, a plataforma preserva a trajetória observacional completa do paciente e expõe inconsistências entre relatos.

Para cada sintoma selecionado em uma sessão de avaliação, o algoritmo aplica três regras principais:

5.1 Restrição por sexo

Sintomas com restrição de sexo contribuem com peso zero quando o paciente é do sexo oposto. Por exemplo, o macroorquidismo é um sintoma aplicável apenas a pacientes masculinos e, portanto, não é computado para pacientes femininos.

5.2 Atenuação para pacientes do sexo feminino

Para pacientes do sexo feminino, o peso de sintomas das categorias cognitiva e comportamental é multiplicado por um fator de atenuação $\alpha = 0,7$. Essa regra busca refletir a manifestação geralmente mais branda da Síndrome do X Frágil em mulheres.

Sintomas físicos não sofrem atenuação

5.3 Soma e comparação com o limiar

A pontuação total é calculada pela soma dos pesos efetivos dos sintomas marcados. Após o cálculo, o sistema compara a pontuação obtida com o limiar τ , configurável pelo administrador.

Quando a pontuação atinge ou supera o limiar definido, o paciente é sinalizado como candidato ao encaminhamento para o teste genético.

Essa abordagem permite que o sistema funcione como uma ferramenta de apoio à decisão, sem substituir a avaliação profissional ou o diagnóstico clínico.

6. Fluxo geral da aplicação

O fluxo da aplicação começa com o cadastro do paciente por um profissional autorizado. Em seguida, são registradas informações básicas, sintomas observados, familiares relacionados e eventuais anexos de evidência visual.

Após o preenchimento das informações, o profissional pode aplicar uma sessão de escore. O backend processa os sintomas selecionados, calcula a pontuação ponderada conforme o sexo do paciente e compara o resultado com o limiar definido pela ONG.

O resultado da avaliação fica registrado no histórico do paciente, permitindo acompanhamento temporal, auditoria das decisões e geração de relatórios.

O administrador pode acompanhar os registros, configurar parâmetros globais e consultar o log de auditoria para verificar ações realizadas dentro da plataforma.

7. Tabelas principais

A modelagem do banco de dados foi organizada para representar o fluxo de triagem do paciente, desde o cadastro inicial até a avaliação dos sintomas, registro de familiares, anexos de evidências e possível encaminhamento para o teste genético. As principais tabelas do sistema são:

users: armazena os usuários da plataforma, incluindo profissionais e administradores. Contém informações como nome, e-mail, hash da senha, papel do usuário no sistema (admin ou profissional), profissão, tipo e número de registro profissional, organização vinculada e estado de ativação da conta.

patients: concentra os dados demográficos e principais informações do paciente, como nome, CPF, sexo, data de nascimento, responsável de referência, localização e situação atual no fluxo de acompanhamento. Os status previstos incluem `em_triagem`, `encaminhado`, `diagnosticado` e `descartado`.

family_members: registra os familiares vinculados ao paciente para fins de rastreo genético. A tabela inclui informações como grau de parentesco, linha familiar materna ou paterna, sexo, condição de diagnóstico conhecida e observações sobre possíveis sintomas apresentados pelo familiar.

symptoms: funciona como um catálogo editável de sintomas utilizados na triagem. Cada sintoma possui uma chave técnica, rótulo apresentado ao usuário, descrição, categoria, peso intrínseco, sexo aplicável e ordem de exibição no formulário.

scores: registra cada sessão individual de aplicação do escore. Cada registro inclui o paciente avaliado, o profissional responsável, o tipo de respondente, o vínculo do respondente, a pontuação total, a pontuação máxima aplicável e o limiar utilizado no momento da avaliação.

score_symptoms: tabela associativa entre os escores e os sintomas selecionados. Ela preserva o peso efetivamente aplicado em cada sintoma, permitindo auditoria futura mesmo que os pesos do catálogo sejam alterados posteriormente.

attachments: armazena referências a arquivos de imagem e vídeo salvos em disco, vinculados ao paciente e, opcionalmente, a uma sessão específica de escore. Esses anexos funcionam como evidências complementares durante a avaliação.

audit_log: registra o histórico cronológico das ações relevantes realizadas no sistema, como criação, alteração, exclusão e login. Cada registro identifica o usuário responsável pela ação, a entidade afetada e os campos modificados em formato JSON.

settings: funciona como um dicionário chave-valor para configurações globais da plataforma, incluindo o limiar do escore e dados de contato da ONG.

8. Requisitos

8.1 Requisitos Funcionais

Identificador	Descrição	Implementação
RF01	Profissionais cadastram pacientes	PatientNew.jsx, POST, /patients
RF02	Administrador cadastra profissionais	Admin.jsx, /admin/professionals, POST
RF03	Registro de sintomas do paciente	ScoreNew.jsx, tabela, score_symptoms
RF04	Registro de novos prontuários	tabela scores

RF05	Login de profissionais	Login.jsx, /auth/login, POST
RF06	Edição de informações do paciente	PATCH /patients/:id
RF07	Edição de atendimentos	PatientDetail.jsx
RF08	Exclusão de atendimentos e pacientes	DELETE /patients/:id
RF09	Bloqueio de acesso não autenticado	middleware authRequired
RF10	Cadastro de paciente com campos obrigatórios	validação em /patients, POST
RF11	Busca de pacientes por CPF	GET /patients/by-cpf/:cpf
RF12	Cálculo automático do score	server/src/score-engine.js
RF13	Consulta cronológica de prontuários	PatientDetail.jsx
RF14	Filtragem de relatórios por período	Reports.jsx, /reports/referrals GET
RF15	Vínculo entre prontuário e profissional autor	Coluna professional_id em scores
RF16	Configuração do limiar de notificação	Admin.jsx, settings
RF17	Gerenciamento de contas de profissionais	Admin.jsx, PATCH /admin/professionals/:id

8.2 Requisitos não funcionais

Identificador	Descrição	Implementação
RNF01	Registro de prontuários	tabela scores
RNF02	Registro de sintomas	score_symptoms
RNF03	Relatórios com prazo determinado	parâmetros from e to em /reports/referrals
RNF04	Acesso a prontuários antigos	PatientDetail.jsx
RNF05	Localização de paciente por CPF	GET /patients/by-cpf/:cpf
RNF06	Mensagem de encaminhamento	resposta da POST /patients/:id/scores e ScoreNew.jsx
RNF07	Relatórios por intervalo temporal	Reports.jsx

RNF08	senha hash	bcrypt com fator 10
RNF09	Restrição a profissionais autorizados	middleware JWT
RNF10	Sigilo dos dados médicos	autenticação e autorização por perfil
RNF11	Agilização do atendimento	busca instantânea por CPF e fluxo único de cadastro
RNF12	Acesso a histórico a qualquer momento	PatientDetail.jsx disponível sob autenticação
RNF13	Registros estruturados	esquema relacional normalizado
RNF14	Histórico de modificações	tabela audit_log

9 Extensões da especificação original

Durante o desenvolvimento, a responsável pelo instituto detalhou pedidos adicionais não contemplados no projeto preliminar, a fim de acolher esses pedidos, as extensões a seguir foram incorporadas:

9.1 Cadastro multiprofissional

A versão inicial restringia o cadastro de pacientes a médicos. A operação do instituto, no entanto, demonstra que outros profissionais com vínculo direto à criança como pedagogos, fonoaudiólogos, psicólogos, psicopedagogos, terapeutas ocupacionais, assistentes sociais e voluntários têm acesso a observações comportamentais que muitas vezes escapam ao contexto do consultório médico. A plataforma agora aceita um conjunto enumerado de profissões e a página de cadastro coleta tipo e número do registro profissional apropriado.

9.2 Múltiplos respondentes por sessão

A noção de respondente foi promovida a atributo de primeira classe da entidade scores. Cada sessão pode ter origem em uma observação direta do profissional, no relato de um familiar cadastrado, em uma ligação telefônica conduzida pelo instituto

ou em campo livre. Esse desenho permite contrastar perspectivas distintas sobre o mesmo paciente, por exemplo, comparando o relato da mãe com o do professor escolar e reconhecendo inconsistências significativas para o diagnóstico diferencial.

9.3 Rastreio familiar

A confirmação genética da Síndrome do X Frágil em um paciente implica probabilidade elevada de portadores entre seus parentes próximos, com transmissão majoritariamente ligada à linha materna. A tabela `family_members` armazena familiares com vínculo, linha, condição de diagnóstico conhecida e sintomas observados, instrumentando a equipe do instituto para acompanhamento longitudinal de famílias.

9.4 Evidência visual

Características como hiperelasticidade nos dedos e morfologia facial são parcialmente observáveis em fotografias e vídeos. A entidade `attachments` permite o upload de imagens e vídeos vinculados ao paciente, opcionalmente associados a um score específico. Esses anexos compõem o material que a equipe analisa antes da decisão de encaminhamento para o teste genético.

10 Implementação e operação

A instalação completa requer somente Node.js 18 ou superior. Os comandos `npm install` e `npm run start` executam, em sequência a instalação das dependências dos dois pacotes, a aplicação das migrations pendentes, a inserção dos dados iniciais (administrador, perfil de demonstração, catálogo de sintomas e configurações padrão), apenas se o arquivo de banco ainda não existir e a inicialização concorrente dos processos de servidor e cliente. Para implantação em produção, serão necessárias substituições.

Repositório:

<https://github.com/lucastiann/EXPERIENCIACRIATIVA.git>